9.3 Heaps

The two strategies for implementing a priority queue ADT in the previous section demonstrate an interesting trade-off. When using an *unsorted* list to store entries, we can perform insertions in $O(1)$ time, but finding or removing an element with minimum key requires an $O(n)$-time loop through the entire collection. In contrast, if using a *sorted* list, we can trivially find or remove the minimum element in $O(1)$ time, but adding a new element to the queue may require $O(n)$ time to restore the sorted order.

In this section, we provide a more efficient realization of a priority queue using a data structure called a ***binary heap***. This data structure allows us to perform both insertions and removals in logarithmic time, which is a significant improvement over the list-based implementations discussed in Section 9.2. The fundamental way the heap achieves this improvement is to use the structure of a binary tree to find a compromise between elements being entirely unsorted and perfectly sorted.

9.3.1 The Heap Data Structure

A heap (see Figure 9.1) is a binary tree T that stores a collection of items at its positions and that satisfies two additional properties: a relational property defined in terms of the way keys are stored in T and a structural property defined in terms of the shape of T itself. The relational property is the following:

***Heap-Order Property*:** In a heap T, for every position p other than the root, the key stored at p is greater than or equal to the key stored at p's parent.

As a consequence of the heap-order property, the keys encountered on a path from the root to a leaf of T are in nondecreasing order. Also, a minimum key is always stored at the root of T. This makes it easy to locate such an item when min or remove_min is called, as it is informally said to be "at the top of the heap" (hence, the name "heap" for the data structure). By the way, the heap data structure defined here has nothing to do with the memory heap (Section 15.1.1) used in the run-time environment supporting a programming language like Python.

For the sake of efficiency, as will become clear later, we want the heap T to have as small a height as possible. We enforce this requirement by insisting that the heap T satisfy an additional structural property—it must be what we term ***complete***.

Complete Binary Tree Property: A heap T with height h is a ***complete*** binary tree if levels $0, 1, 2, \ldots, h-1$ of T have the maximum number of nodes possible (namely, level i has 2^i nodes, for $0 \leq i \leq h-1$) and the remaining nodes at level h reside in the leftmost possible positions at that level.

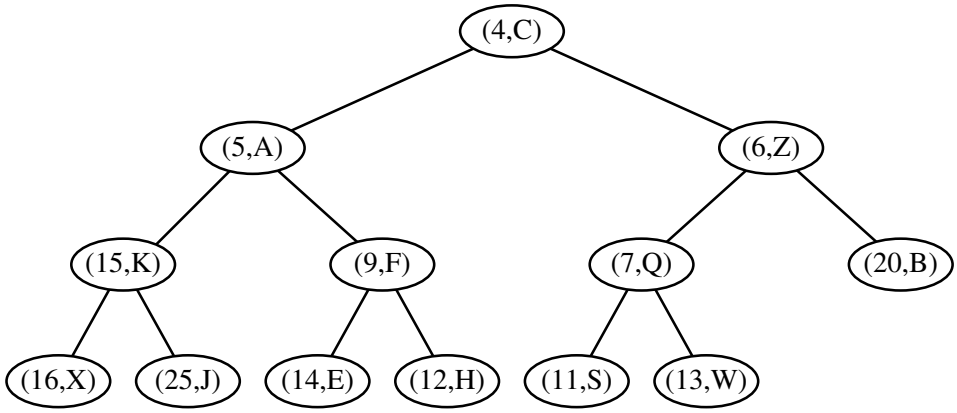


Figure 9.1: Example of a heap storing 13 entries with integer keys. The last position is the one storing entry $(13,W)$.

The tree in Figure 9.1 is complete because levels 0, 1, and 2 are full, and the six nodes in level 3 are in the six leftmost possible positions at that level. In formalizing what we mean by the leftmost possible positions, we refer to the discussion of ***level numbering*** from Section 8.3.2, in the context of an array-based representation of a binary tree. (In fact, in Section 9.3.3 we will discuss the use of an array to represent a heap.) A complete binary tree with n elements is one that has positions with level numbering 0 through $n-1$. For example, in an array-based representation of the above tree, its 13 entries would be stored consecutively from $A[0]$ to $A[12]$.

The Height of a Heap

Let h denote the height of T. Insisting that T be complete also has an important consequence, as shown in Proposition 9.2.

Proposition 9.2: *A heap T storing n entries has height $h = \lfloor \log n \rfloor$.*

Justification: From the fact that T is complete, we know that the number of nodes in levels 0 through $h-1$ of T is precisely $1+2+4+\cdots+2^{h-1} = 2^h - 1$, and that the number of nodes in level h is at least 1 and at most 2^h. Therefore

$$n \geq 2^h - 1 + 1 = 2^h \qquad \text{and} \qquad n \leq 2^h - 1 + 2^h = 2^{h+1} - 1.$$

By taking the logarithm of both sides of inequality $2^h \leq n$, we see that height $h \leq \log n$. By rearranging terms and taking the logarithm of both sides of inequality $n \leq 2^{h+1} - 1$, we see that $\log(n+1) - 1 \leq h$. Since h is an integer, these two inequalities imply that $h = \lfloor \log n \rfloor$. ■

9.3.2 Implementing a Priority Queue with a Heap

Proposition 9.2 has an important consequence, for it implies that if we can perform update operations on a heap in time proportional to its height, then those operations will run in logarithmic time. Let us therefore turn to the problem of how to efficiently perform various priority queue methods using a heap.

We will use the composition pattern from Section 9.2.1 to store key-value pairs as items in the heap. The len and is_empty methods can be implemented based on examination of the tree, and the min operation is equally trivial because the heap property assures that the element at the root of the tree has a minimum key. The interesting algorithms are those for implementing the add and remove_min methods.

Adding an Item to the Heap

Let us consider how to perform add(k,v) on a priority queue implemented with a heap T. We store the pair (k, v) as an item at a new node of the tree. To maintain the ***complete binary tree property***, that new node should be placed at a position p just beyond the rightmost node at the bottom level of the tree, or as the leftmost position of a new level, if the bottom level is already full (or if the heap is empty).

Up-Heap Bubbling After an Insertion

After this action, the tree T is complete, but it may violate the ***heap-order property***. Hence, unless position p is the root of T (that is, the priority queue was empty before the insertion), we compare the key at position p to that of p's parent, which we denote as q. If key $k_p \geq k_q$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p < k_q$, then we need to restore the heap-order property, which can be locally achieved by swapping the entries stored at positions p and q. (See Figure 9.2c and d.) This swap causes the new item to move up one level. Again, the heap-order property may be violated, so we repeat the process, going up in T until no violation of the heap-order property occurs. (See Figure 9.2e and h.)

The upward movement of the newly inserted entry by means of swaps is conventionally called ***up-heap bubbling***. A swap either resolves the violation of the heap-order property or propagates it one level up in the heap. In the worst case, up-heap bubbling causes the new entry to move all the way up to the root of heap T. Thus, in the worst case, the number of swaps performed in the execution of method add is equal to the height of T. By Proposition 9.2, that bound is $\lfloor \log n \rfloor$.

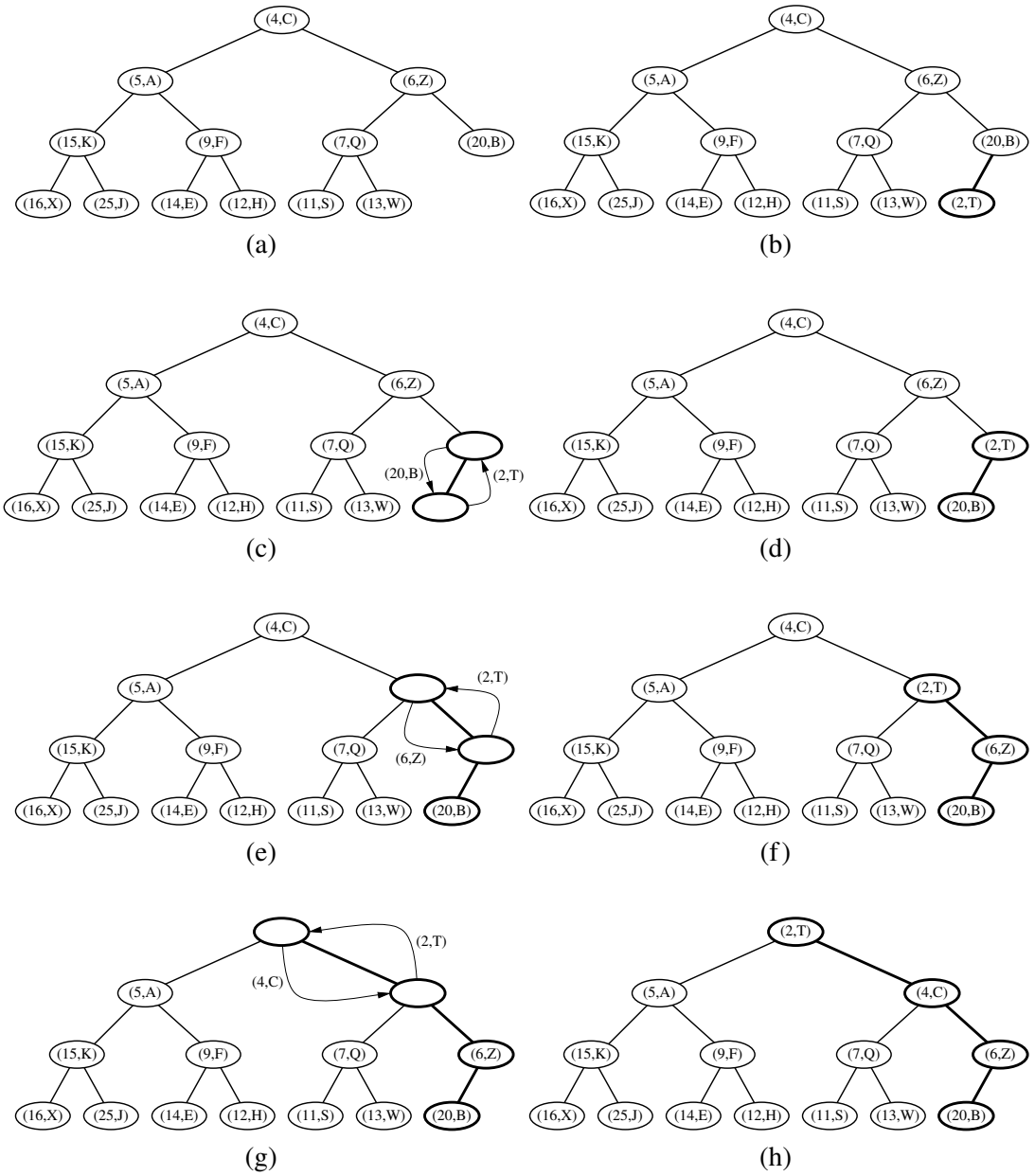


Figure 9.2: Insertion of a new entry with key 2 into the heap of Figure 9.1: (a) initial heap; (b) after performing operation add; (c and d) swap to locally restore the partial order property; (e and f) another swap; (g and h) final swap.

Removing the Item with Minimum Key

Let us now turn to method remove_min of the priority queue ADT. We know that an entry with the smallest key is stored at the root r of T (even if there is more than one entry with smallest key). However, in general we cannot simply delete node r, because this would leave two disconnected subtrees.

Instead, we ensure that the shape of the heap respects the ***complete binary tree property*** by deleting the leaf at the *last* position p of T, defined as the rightmost position at the bottommost level of the tree. To preserve the item from the last position p, we copy it to the root r (in place of the item with minimum key that is being removed by the operation). Figure 9.3a and b illustrates an example of these steps, with minimal item $(4,C)$ being removed from the root and replaced by item $(13,W)$ from the last position. The node at the last position is removed from the tree.

Down-Heap Bubbling After a Removal

We are not yet done, however, for even though T is now complete, it likely violates the heap-order property. If T has only one node (the root), then the heap-order property is trivially satisfied and the algorithm terminates. Otherwise, we distinguish two cases, where p initially denotes the root of T:

- If p has no right child, let c be the left child of p.
- Otherwise (p has both children), let c be a child of p with minimal key.

If key $k_p \leq k_c$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p > k_c$, then we need to restore the heap-order property. This can be locally achieved by swapping the entries stored at p and c. (See Figure 9.3c and d.) It is worth noting that when p has two children, we intentionally consider the *smaller* key of the two children. Not only is the key of c smaller than that of p, it is at least as small as the key at c's sibling. This ensures that the heap-order property is locally restored when that smaller key is promoted above the key that had been at p and that at c's sibling.

Having restored the heap-order property for node p relative to its children, there may be a violation of this property at c; hence, we may have to continue swapping down T until no violation of the heap-order property occurs. (See Figure 9.3e–h.) This downward swapping process is called ***down-heap bubbling***. A swap either resolves the violation of the heap-order property or propagates it one level down in the heap. In the worst case, an entry moves all the way down to the bottom level. (See Figure 9.3.) Thus, the number of swaps performed in the execution of method remove_min is, in the worst case, equal to the height of heap T, that is, it is $\lfloor \log n \rfloor$ by Proposition 9.2.

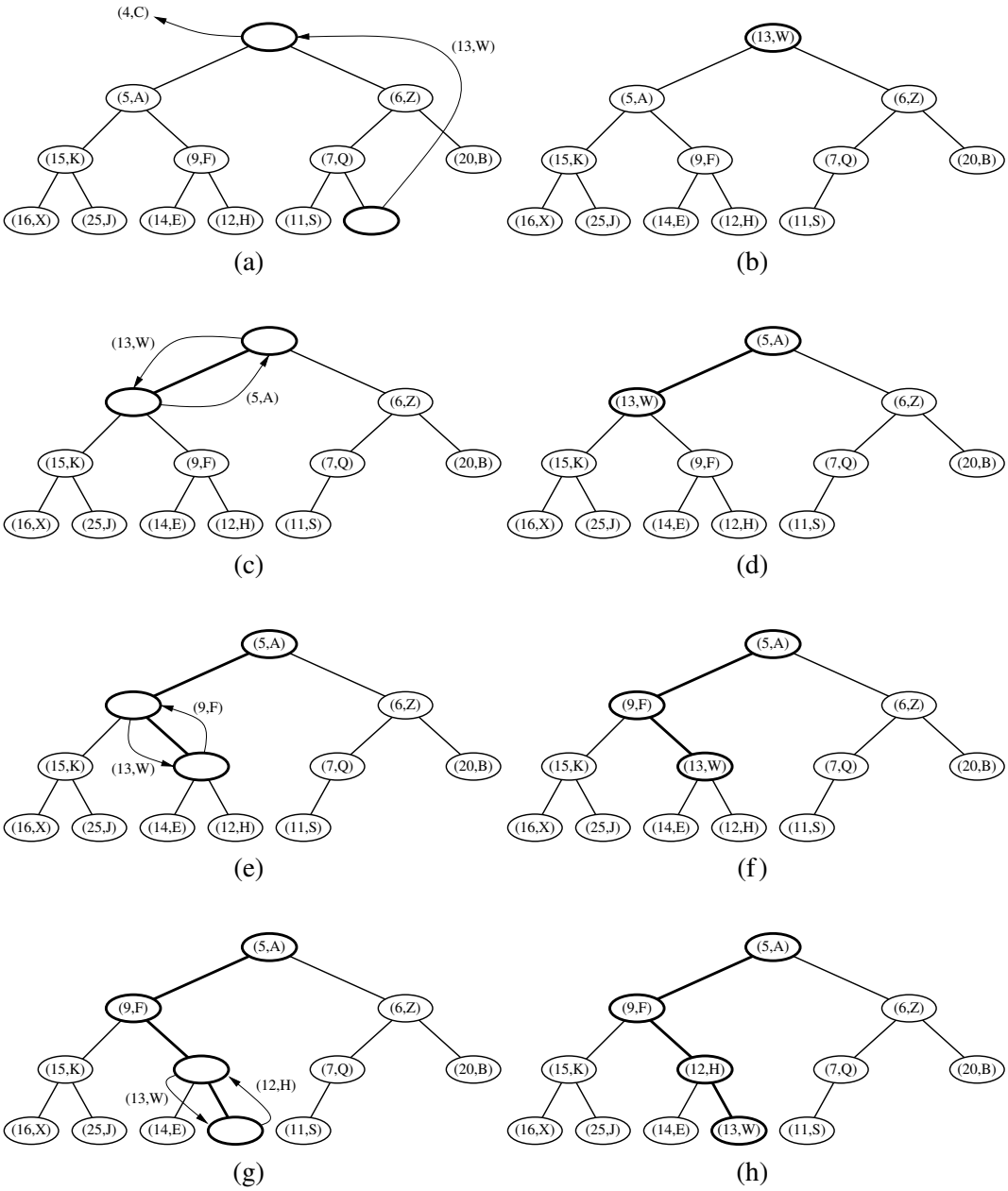


Figure 9.3: Removal of the entry with the smallest key from a heap: (a and b) deletion of the last node, whose entry gets stored into the root; (c and d) swap to locally restore the heap-order property; (e and f) another swap; (g and h) final swap.

9.3.3 Array-Based Representation of a Complete Binary Tree

The array-based representation of a binary tree (Section 8.3.2) is especially suitable for a complete binary tree T. We recall that in this implementation, the elements of T are stored in an array-based list A such that the element at position p in T is stored in A with index equal to the level number $f(p)$ of p, defined as follows:

- If p is the root of T, then $f(p) = 0$.
- If p is the left child of position q, then $f(p) = 2f(q) + 1$.
- If p is the right child of position q, then $f(p) = 2f(q) + 2$.

With this implementation, the elements of T have contiguous indices in the range $[0, n-1]$ and the last position of T is always at index $n-1$, where n is the number of positions of T. For example, Figure 9.4 illustrates the array-based representation of the heap structure originally portrayed in Figure 9.1.

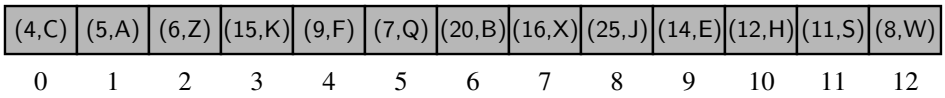


Figure 9.4: An array-based representation of the heap from Figure 9.1.

Implementing a priority queue using an array-based heap representation allows us to avoid some complexities of a node-based tree structure. In particular, the add and remove_min operations of a priority queue both depend on locating the last index of a heap of size n. With the array-based representation, the last position is at index $n-1$ of the array. Locating the last position of a complete binary tree implemented with a linked structure requires more effort. (See Exercise C-9.34.)

If the size of a priority queue is not known in advance, use of an array-based representation does introduce the need to dynamically resize the array on occasion, as is done with a Python list. The space usage of such an array-based representation of a complete binary tree with n nodes is $O(n)$, and the time bounds of methods for adding or removing elements become ***amortized***. (See Section 5.3.1.)

9.3.4 Python Heap Implementation

We provide a Python implementation of a heap-based priority queue in Code Fragments 9.4 and 9.5. We use an array-based representation, maintaining a Python list of item composites. Although we do not formally use the binary tree ADT, Code Fragment 9.4 includes nonpublic utility functions that compute the level numbering of a parent or child of another. This allows us to describe the rest of our algorithms using tree-like terminology of *parent*, *left*, and *right*. However, the relevant variables are integer indexes (not "position" objects). We use recursion to implement the repetition in the _upheap and _downheap utilities.

```
1  class HeapPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with a binary heap."""
3    #------------------------------ nonpublic behaviors ------------------------------
4    def _parent(self, j):
5      return (j-1) // 2
6
7    def _left(self, j):
8      return 2*j + 1
9
10   def _right(self, j):
11     return 2*j + 2
12
13   def _has_left(self, j):
14     return self._left(j) < len(self._data)      # index beyond end of list?
15
16   def _has_right(self, j):
17     return self._right(j) < len(self._data)     # index beyond end of list?
18
19   def _swap(self, i, j):
20     """Swap the elements at indices i and j of array."""
21     self._data[i], self._data[j] = self._data[j], self._data[i]
22
23   def _upheap(self, j):
24     parent = self._parent(j)
25     if j > 0 and self._data[j] < self._data[parent]:
26       self._swap(j, parent)
27       self._upheap(parent)                      # recur at position of parent
28
29   def _downheap(self, j):
30     if self._has_left(j):
31       left = self._left(j)
32       small_child = left                        # although right may be smaller
33       if self._has_right(j):
34         right = self._right(j)
35         if self._data[right] < self._data[left]:
36           small_child = right
37       if self._data[small_child] < self._data[j]:
38         self._swap(j, small_child)
39         self._downheap(small_child)             # recur at position of small child
```

Code Fragment 9.4: An implementation of a priority queue using an array-based heap (continued in Code Fragment 9.5). The extends the PriorityQueueBase class from Code Fragment 9.1.

```
40    #------------------------------ public behaviors ------------------------------
41    def __init__(self):
42      """Create a new empty Priority Queue."""
43      self._data = [ ]
44
45    def __len__(self):
46      """Return the number of items in the priority queue."""
47      return len(self._data)
48
49    def add(self, key, value):
50      """Add a key-value pair to the priority queue."""
51      self._data.append(self._Item(key, value))
52      self._upheap(len(self._data) - 1)          # upheap newly added position
53
54    def min(self):
55      """Return but do not remove (k,v) tuple with minimum key.
56
57      Raise Empty exception if empty.
58      """
59      if self.is_empty( ):
60        raise Empty('Priority queue is empty.')
61      item = self._data[0]
62      return (item._key, item._value)
63
64    def remove_min(self):
65      """Remove and return (k,v) tuple with minimum key.
66
67      Raise Empty exception if empty.
68      """
69      if self.is_empty( ):
70        raise Empty('Priority queue is empty.')
71      self._swap(0, len(self._data) - 1)         # put minimum item at the end
72      item = self._data.pop( )                   # and remove it from the list;
73      self._downheap(0)                          # then fix new root
74      return (item._key, item._value)
```

Code Fragment 9.5: An implementation of a priority queue using an array-based heap (continued from Code Fragment 9.4).

9.3.5 Analysis of a Heap-Based Priority Queue

Table 9.3 shows the running time of the priority queue ADT methods for the heap implementation of a priority queue, assuming that two keys can be compared in $O(1)$ time and that the heap T is implemented with an array-based or linked-based tree representation.

In short, each of the priority queue ADT methods can be performed in $O(1)$ or in $O(\log n)$ time, where n is the number of entries at the time the method is executed. The analysis of the running time of the methods is based on the following:

- The heap T has n nodes, each storing a reference to a key-value pair.
- The height of heap T is $O(\log n)$, since T is complete (Proposition 9.2).
- The min operation runs in $O(1)$ because the root of the tree contains such an element.
- Locating the last position of a heap, as required for add and remove_min, can be performed in $O(1)$ time for an array-based representation, or $O(\log n)$ time for a linked-tree representation. (See Exercise C-9.34.)
- In the worst case, up-heap and down-heap bubbling perform a number of swaps equal to the height of T.

Operation	Running Time
len(P), P.is_empty( )	$O(1)$
P.min( )	$O(1)$
P.add( )	$O(\log n)^*$
P.remove_min( )	$O(\log n)^*$

*amortized, if array-based

Table 9.3: Performance of a priority queue, P, realized by means of a heap. We let n denote the number of entries in the priority queue at the time an operation is executed. The space requirement is $O(n)$. The running time of operations min and remove_min are amortized for an array-based representation, due to occasional resizing of a dynamic array; those bounds are worst case with a linked tree structure.

We conclude that the heap data structure is a very efficient realization of the priority queue ADT, independent of whether the heap is implemented with a linked structure or an array. The heap-based implementation achieves fast running times for both insertion and removal, unlike the implementations that were based on using an unsorted or sorted list.

9.3.6 Bottom-Up Heap Construction ⋆

If we start with an initially empty heap, n successive calls to the add operation will run in $O(n \log n)$ time in the worst case. However, if all n key-value pairs to be stored in the heap are given in advance, such as during the first phase of the heap-sort algorithm, there is an alternative ***bottom-up*** construction method that runs in $O(n)$ time. (Heap-sort, however, still requires $\Theta(n \log n)$ time because of the second phase in which we repeatedly remove the remaining element with smallest key.)

In this section, we describe the bottom-up heap construction, and provide an implementation that can be used by the constructor of a heap-based priority queue.

For simplicity of exposition, we describe this bottom-up heap construction assuming the number of keys, n, is an integer such that $n = 2^{h+1} - 1$. That is, the heap is a complete binary tree with every level being full, so the heap has height $h = \log(n+1) - 1$. Viewed nonrecursively, bottom-up heap construction consists of the following $h + 1 = \log(n+1)$ steps:

1. In the first step (see Figure 9.5b), we construct $(n+1)/2$ elementary heaps storing one entry each.
2. In the second step (see Figure 9.5c–d), we form $(n+1)/4$ heaps, each storing three entries, by joining pairs of elementary heaps and adding a new entry. The new entry is placed at the root and may have to be swapped with the entry stored at a child to preserve the heap-order property.
3. In the third step (see Figure 9.5e–f), we form $(n+1)/8$ heaps, each storing 7 entries, by joining pairs of 3-entry heaps (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

   ⋮

i. In the generic i^{th} step, $2 \leq i \leq h$, we form $(n+1)/2^i$ heaps, each storing $2^i - 1$ entries, by joining pairs of heaps storing $(2^{i-1} - 1)$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

   ⋮

$h+1$. In the last step (see Figure 9.5g–h), we form the final heap, storing all the n entries, by joining two heaps storing $(n-1)/2$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

We illustrate bottom-up heap construction in Figure 9.5 for $h = 3$.

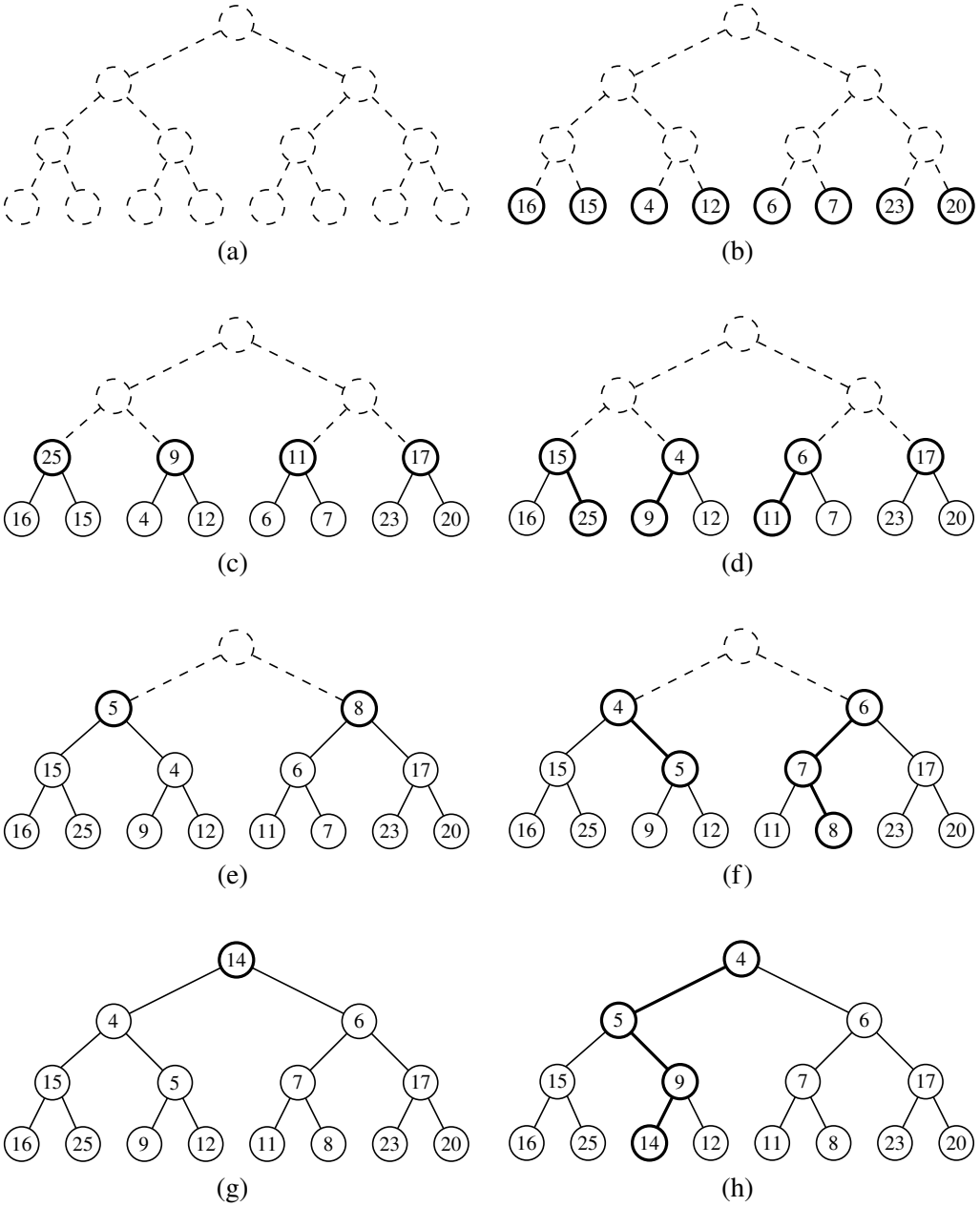


Figure 9.5: Bottom-up construction of a heap with 15 entries: (a and b) we begin by constructing 1-entry heaps on the bottom level; (c and d) we combine these heaps into 3-entry heaps, and then (e and f) 7-entry heaps, until (g and h) we create the final heap. The paths of the down-heap bubblings are highlighted in (d, f, and h). For simplicity, we only show the key within each node instead of the entire entry.

Python Implementation of a Bottom-Up Heap Construction

Implementing a bottom-up heap construction is quite easy, given the existence of a "down-heap" utility function. The "merging" of two equally sized heaps that are subtrees of a common position p, as described in the opening of this section, can be accomplished simply by down-heaping p's entry. For example, that is what happened to the key 14 in going from Figure 9.5(f) to (g).

With our array-based representation of a heap, if we initially store all n items in arbitrary order within the array, we can implement the bottom-up heap construction process with a single loop that makes a call to _downheap from each position of the tree, as long as those calls are ordered starting with the *deepest* level and ending with the root of the tree. In fact, that loop can start with the deepest nonleaf, since there is no effect when down-heap is called at a leaf position.

In Code Fragment 9.6, we augment the original HeapPriorityQueue class from Section 9.3.4 to provide support for the bottom-up construction of an initial collection. We introduce a nonpublic utility method, _heapify, that calls _downheap on each nonleaf position, beginning with the deepest and concluding with a call at the root of the tree. We have redesigned the constructor of the class to accept an optional parameter that can be any sequence of (k,v) tuples. Rather than initializing self._data to an empty list, we use a list comprehension syntax (see Section 1.9.2) to create an initial list of item composites based on the given contents. We declare an empty sequence as the default parameter value so that the default syntax HeapPriorityQueue( ) continues to result in an empty priority queue.

```
def __init__(self, contents=( )):
  """Create a new priority queue.

  By default, queue will be empty. If contents is given, it should be as an
  iterable sequence of (k,v) tuples specifying the initial contents.
"""
  self._data = [ self._Item(k,v) for k,v in contents ]      # empty by default
  if len(self._data) > 1:
    self._heapify( )

def _heapify(self):
  start = self._parent(len(self) - 1)       # start at PARENT of last leaf
  for j in range(start, -1, -1):            # going to and including the root
    self._downheap(j)
```

Code Fragment 9.6: Revision to the HeapPriorityQueue class of Code Fragments 9.4 and 9.5 to support a linear-time construction given an initial sequence of entries.

Asymptotic Analysis of Bottom-Up Heap Construction

Bottom-up heap construction is asymptotically faster than incrementally inserting n keys into an initially empty heap. Intuitively, we are performing a single down-heap operation at each position in the tree, rather than a single up-heap operation from each. Since more nodes are closer to the bottom of a tree than the top, the sum of the downward paths is linear, as shown in the following proposition.

Proposition 9.3: *Bottom-up construction of a heap with n entries takes $O(n)$ time, assuming two keys can be compared in $O(1)$ time.*

Justification: The primary cost of the construction is due to the down-heap steps performed at each nonleaf position. Let π_v denote the path of T from nonleaf node v to its "inorder successor" leaf, that is, the path that starts at v, goes to the right child of v, and then goes down leftward until it reaches a leaf. Although, π_v is not necessarily the path followed by the down-heap bubbling step from v, the length $\|\pi_v\|$ (its number of edges) is proportional to the height of the subtree rooted at v, and thus a bound on the complexity of the down-heap operation at v. We can bound the total running time of the bottom-up heap construction algorithm based on the sum of the sizes of paths, $\sum_v \|\pi_v\|$. For intuition, Figure 9.6 illustrates the justification "visually," marking each edge with the label of the nonleaf node v whose path π_v contains that edge.

We claim that the paths π_v for all nonleaf v are edge-disjoint, and thus the sum of the path lengths is bounded by the number of total edges in the tree, hence $O(n)$. To show this, we consider what we term "right-leaning" and "left-leaning" edges (i.e., those going from a parent to a right, respectively left, child). A particular right-leaning edge e can only be part of the path π_v for node v that is the parent in the relationship represented by e. Left-leaning edges can be partitioned by considering the leaf that is reached if continuing down leftward until reaching a leaf. Each nonleaf node only uses left-leaning edges in the group leading to that nonleaf node's inorder successor. Since each nonleaf node must have a different inorder successor, no two such paths can contain the same left-leaning edge. We conclude that the bottom-up construction of heap T takes $O(n)$ time. ■

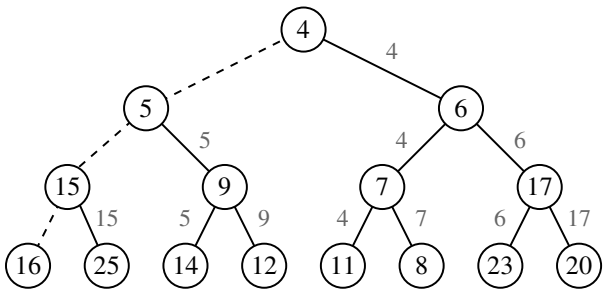


Figure 9.6: Visual justification of the linear running time of bottom-up heap construction. Each edge e is labeled with a node v for which π_v contains e (if any).

9.3.7 Python's heapq Module

Python's standard distribution includes a heapq module that provides support for heap-based priority queues. That module does not provide any priority queue class; instead it provides functions that allow a standard Python list to be managed as a heap. Its model is essentially the same as our own, with n elements stored in list cells $L[0]$ through $L[n-1]$, based on the level-numbering indices with the smallest element at the root in $L[0]$. We note that heapq does not separately manage associated values; elements serve as their own key.

The heapq module supports the following functions, all of which presume that existing list L satisfies the heap-order property prior to the call:

heappush(L, e): Push element e onto list L and restore the heap-order property. The function executes in $O(\log n)$ time.

heappop(L): Pop and return the element with smallest value from list L, and reestablish the heap-order property. The operation executes in $O(\log n)$ time.

heappushpop(L, e): Push element e on list L and then pop and return the smallest item. The time is $O(\log n)$, but it is slightly more efficient than separate calls to push and pop because the size of the list never changes. If the newly pushed element becomes the smallest, it is immediately returned. Otherwise, the new element takes the place of the popped element at the root and a down-heap is performed.

heapreplace(L, e): Similar to heappushpop, but equivalent to the pop being performed before the push (in other words, the new element cannot be returned as the smallest). Again, the time is $O(\log n)$, but it is more efficient that two separate operations.

The module supports additional functions that operate on sequences that do not previously satisfy the heap-order property.

heapify(L): Transform unordered list to satisfy the heap-order property. This executes in $O(n)$ time by using the bottom-up construction algorithm.

nlargest(k, iterable): Produce a list of the k largest values from a given iterable. This can be implemented to run in $O(n + k \log n)$ time, where we use n to denote the length of the iterable (see Exercise C-9.42).

nsmallest(k, iterable): Produce a list of the k smallest values from a given iterable. This can be implemented to run in $O(n + k \log n)$ time, using similar technique as with nlargest.